SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – APPLICATION BASED QUESTIONS

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Application Based Questions, In-class
Weightage:	_____ 40% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Dinesh R	**Reg. No.:**	192421117
Section / Batch:	B.Tech-IT	**Date:**	20/10/2026

Code of Conduct

I Dinesh R-192421117 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Dinesh R

Instructions

- Read each question carefully before answering (in-Class Assessment)
- Each question is a real-world application scenario. Identify the AI technique that fits, apply it step by step, and justify every decision.
- Questions are drawn from Informed Search (Greedy, A*, heuristics), Local Search and Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Show intermediate tables (open lists, domains, α/β values) and state the final answer clearly.
- Marks will be awarded using the Application Based Questions rubric: understanding of problem context, application of concepts, problem-solving approach, accuracy of solution and clarity.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions (admissibility, consistency)	**Q1**
2. Local Search, Online Search & Constraint Satisfaction	Local search and optimization problems, online search agent, CSP (MRV, degree heuristic, forward checking)	**Q2, Q3**
3. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q4**

Rubrics for Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Problem Context	**20%**	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	**30%**	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	**20%**	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	**20%**	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	**10%**	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context (2)	Application of Concepts (3)	Problem-Solving Approach (2)	Accuracy of Solution (2)	Clarity (1)	Sub. Total (10)	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1: Campus Delivery Robot – Greedy Best-First vs A*

Topic: Informed search strategies, heuristic functions | 15 minutes

A campus delivery robot must carry a parcel from the Store (S) to the Hostel (G). The one-way road network and the robot's heuristic estimate h(n) of the remaining travel cost to G (in minutes) are given below.

From	To	Cost
S	A	1
S	B	4
A	C	2
A	D	5
B	G	5
C	D	1
D	G	2

Node	S	A	B	C	D	G
h(n)	6	5	3	3	2	0

Assume ties are broken alphabetically and that a node is re-opened if a cheaper path to it is found.

(a) Apply Greedy Best-First Search from S to G. Write the order in which nodes are expanded, the path returned and its total cost.

(b) Apply A* search. Show the open list with g(n), h(n) and f(n) after each expansion, the order of expansion, and the final path and cost.

(c) Check whether h(n) is admissible and consistent by verifying the required condition for every node and every road.

(d) Compare the outputs of (a) and (b). Explain why Greedy Best-First Search returned a different result and which algorithm you would deploy for the robot, with justification.

(a) Greedy Best-First Search

Given:

Road	**Cost**
S → A	1
S → B	4
A → C	2
A → D	5
B → G	5
C → D	1

Road	Cost
D → G	2

Heuristic values:

Node	S	A	B	C	D	G
h(n)	6	5	3	3	2	0

Greedy Best-First Search selects the node with the smallest heuristic value, **h(n)**.

Step-by-step

1. Start at S. Expand S.
   - A: h(A) = 5
   - B: h(B) = 3
   - Select B.
2. Expand B.
   - G: h(G) = 0
   - Select G.
3. G is the goal, so stop.

Answer:

- Expansion order: S → B → G
- Final path: S → B → G
- Total cost = 4 + 5 = **9 minutes**

(b) A* Search

A* calculates:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$= actual cost from the start.
- $h(n)$= estimated remaining cost.
- $f(n)$= estimated total cost.

Step-by-step OPEN list

Initially, OPEN contains S.

Step	Expanded node	OPEN list after expansion (node: g, h, f)
1	S	A: (1, 5, 6), B: (4, 3, 7)
2	A	B: (4, 3, 7), C: (3, 3, 6), D: (6, 2, 8)
3	C	B: (4, 3, 7), D: (4, 2, 6)
4	D	B: (4, 3, 7), G: (6, 0, 6)

Step	Expanded node	OPEN list after expansion (node: g, h, f)
5	G	Stop; goal reached

When D is expanded, the route through D gives G a cost of 6.

Answer:

- Expansion order: S → A → C → D → G
- Final path: S → A → C → D → G
- Total cost = 1 + 2 + 1 + 2 = **6 minutes**

(c) Check whether the heuristic is admissible and consistent

1. Admissibility

A heuristic is admissible if:

$$h(n) \leq h^*(n)$$

Here, $h^*(n)$is the actual shortest distance from node $n$to G.

Calculate the actual shortest distances:

Node	Actual shortest distance to G	h(n)	Admissible?
S	6	6	Yes
A	5	5	Yes
B	5	3	Yes
C	3	3	Yes
D	2	2	Yes
G	0	0	Yes

Since $h(n)$never exceeds the actual shortest distance, **the heuristic is admissible**.

2. Consistency

A heuristic is consistent if, for every edge $n \to n'$,

$$h(n) \leq c(n, n') + h(n')$$

Check each road:

Edge	Check	Result
S → A	6 ≤ 1 + 5 = 6	Consistent
S → B	6 ≤ 4 + 3 = 7	Consistent
A → C	5 ≤ 2 + 3 = 5	Consistent
A → D	5 ≤ 5 + 2 = 7	Consistent
B → G	3 ≤ 5 + 0 = 5	Consistent
C → D	3 ≤ 1 + 2 = 3	Consistent

D → G	$2 \leq 2 + 0 = 2$	Consistent

Conclusion: The heuristic is both admissible and consistent.

(d) Compare Greedy Best-First Search and A*

Feature	Greedy Best-First Search	A* Search
Evaluation function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Considers path cost?	No	Yes
Returned path	S → B → G	S → A → C → D → G
Total cost	9 minutes	6 minutes
Optimal solution guaranteed?	No	Yes, with an admissible heuristic under the standard graph-search conditions

Explanation:

Greedy Best-First Search chooses B because its heuristic value is smaller than A's. However, it ignores the cost already incurred and the total cost of the route.

A* considers both the path cost and estimated remaining cost, allowing it to find the cheaper route.

Recommendation: Deploy A* because it finds the shortest route for the given network and uses both actual and estimated travel costs.

Question 2: Wi-Fi Access Point Placement – Local Search and Online Search

Topic: Local search algorithms and optimization problems, online search agent | 15 minutes

A technician wants to place a Wi-Fi access point at the best position along a corridor with 10 candidate positions x = 0 to 9. The measured signal quality f(x) (higher is better) is:

x	0	1	2	3	4	5	6	7	8	9
f(x)	2	5	9	6	4	7	12	10	8	3

A move shifts the access point to an adjacent position ($x - 1$ or $x + 1$). Steepest-ascent hill climbing moves to the better neighbour and stops when no neighbour is strictly better.

(a) Run steepest-ascent hill climbing from the start positions x = 0, x = 4 and x = 9. Show each step and the final position and signal quality.

(b) Identify all local maxima and the global maximum. If the start position is chosen uniformly at random from the 10 positions, find the probability that one run reaches the global maximum, the probability that at least one of 3 random-restart runs does, and the expected number of runs needed.

(c) A simulated annealing search is at x = 2 and proposes the move to x = 3. Compute the acceptance probability e^(Δ/T) at T = 2 and at T = 1, and explain what this does to the chance of escaping the local maximum as T falls.

(d) The same search is later done by a robot that cannot see the whole corridor, can only measure f(x) where it stands, and cannot undo exploration for free. Explain why this is an online search problem, and state what the agent must remember to avoid cycling.

(a) Steepest-ascent hill climbing

The algorithm moves to a strictly better adjacent position and stops when neither neighbor is better.

Starting at x = 0

- $x = 0, f(0) = 2$
- Move to $x = 1, f(1) = 5$
- Move to $x = 2, f(2) = 9$
- At $x = 2$, the neighbors have values 5 and 6. Neither is greater than 9.
- Stop.

Final position: $x = 2$

Signal quality: 9

Starting at x = 4

- $x = 4, f(4) = 4$
- Move to $x = 5, f(5) = 7$
- Move to $x = 6, f(6) = 12$
- At $x = 6$, the neighbors have values 7 and 10. Neither is greater than 12.

- Stop.

Final position: $x = 6$

Signal quality: 12

Starting at x = 9

- $x = 9, f(9) = 3$
- Move to $x = 8, f(8) = 8$
- Move to $x = 7, f(7) = 10$
- Move to $x = 6, f(6) = 12$
- Stop at $x = 6$.

Final position: $x = 6$

Signal quality: 12

Summary

Start position	Search path	Final position	Final signal
0	$0 \rightarrow 1 \rightarrow 2$	2	9
4	$4 \rightarrow 5 \rightarrow 6$	6	12
9	$9 \rightarrow 8 \rightarrow 7 \rightarrow 6$	6	12

(b) Local maxima, global maximum, and probabilities

A local maximum has a value greater than the values at its adjacent positions.

Local maxima

- At $x = 2, f(2) = 9$, greater than $f(1) = 5$and $f(3) = 6$.
- At $x = 6, f(6) = 12$, greater than $f(5) = 7$and $f(7) = 10$.

Therefore:

- Local maxima: **x = 2 and x = 6**
- Global maximum: **x = 6, with signal quality 12**

1. Probability that one random run reaches the global maximum

The starting position is selected uniformly from 10 positions.

The algorithm reaches the global maximum from starting positions:

$$\{3, 4, 5, 6, 7, 8, 9\}$$

That is 7 starting positions.

Therefore,

$$P(\text{success}) = \frac{7}{10} = \boxed{0.7}$$

Answer: 70%.

2. Probability that at least one of three random-restart runs succeeds

The probability that one run fails is:

$$P(\text{failure}) = 1 - \frac{7}{10} = \frac{3}{10}$$

Assuming independent random starts, the probability that all three runs fail is:

$$P(\text{all fail}) = \left(\frac{3}{10}\right)^3 = 0.027$$

Thus,

$$P(\text{at least one success}) = 1 - 0.027$$
$$\boxed{0.973 = 97.3\%}$$

3. Expected number of runs needed

For independent runs with success probability $p = 0.7$, the expected number of runs until the first success follows a geometric distribution:

$$E[N] = \frac{1}{p}$$
$$E[N] = \frac{1}{0.7} = \boxed{1.43 \text{ runs}}$$

(c) Simulated annealing

The search moves from $x = 2$to $x = 3$.

Given:

$$f(2) = 9,\ f(3) = 6$$

The change in signal quality is:

$$\Delta = f(3) - f(2) = 6 - 9 = -3$$

For a worse move, the acceptance probability is:

$$P = e^{\Delta/T}$$

At T = 2

$$P = e^{-3/2}$$
$$\boxed{P \approx 0.2231 = 22.31\%}$$

At T = 1

$$P = e^{-3}$$
$$\boxed{P \approx 0.0498 = 4.98\%}$$

Explanation

Temperature	Acceptance probability
$T = 2$	22.31%
$T = 1$	4.98%

As temperature decreases, the probability of accepting a worse move decreases.

At higher temperatures, simulated annealing is more willing to accept worse moves, helping it escape local maxima. At lower temperatures, it becomes more selective and increasingly favors improvements.

(d) Online search problem

This is an **online search problem** because the robot:

1. Cannot observe the entire corridor in advance.
2. Can measure signal quality only at its current position.
3. Must move through the corridor to discover neighboring positions.
4. Cannot undo exploration without incurring movement costs.

Question 3: Exam Timetable Scheduling – Constraint Satisfaction Problem

Topic: Constraint satisfaction problem (MRV, degree heuristic, forward checking) | 15 minutes

Five exams A, B, C, D and E must each be given one slot from {1, 2, 3}. Two exams that share students must not be in the same slot. The pairs sharing students are: A–B, A–C, A–D, B–C, B–D, C–E, D–E. Because of invigilator availability, exam A can only be in slots 1 or 2, and exam E only in slots 1 or 2.

Exam	Initial domain
A	{1, 2}
B	{1, 2, 3}
C	{1, 2, 3}
D	{1, 2, 3}
E	{1, 2}

Rules for the search: choose the variable using Minimum Remaining Values (MRV), break ties with the degree heuristic, then alphabetically; try values in ascending order; use forward checking after every assignment.

(a) Draw the constraint graph (list its edges) and compute the degree of every exam.

(b) Select the first exam to assign using MRV and the degree heuristic, justify the choice, assign its first value, and show the domains of all exams after forward checking.

(c) Continue the search with the same rules, showing the chosen exam, its value and the updated domains at each step, until a complete assignment is found. Verify it against every constraint.

(d) Interpret: explain how MRV, the degree heuristic and forward checking reduced the search effort here, and what forward checking would signal if a domain became empty.

(a) Constraint graph and degrees

The degree of a variable is the number of other variables connected to it.

Exam	Connected exams	Degree
A	B, C, D	3
B	A, C, D	3
C	A, B, E	3
D	A, B, E	3
E	C, D	2

Answer: The constraint graph has seven edges, and the degrees are:

$$\boxed{\deg(A) = 3, \deg(B) = 3, \deg(C) = 3, \deg(D) = 3, \deg(E) = 2}$$

(b) Select the first exam using MRV and degree heuristic

Step 1: Apply MRV

Minimum Remaining Values selects the variable with the smallest domain.

- A has 2 values.
- B has 3 values.
- C has 3 values.
- D has 3 values.
- E has 2 values.

The candidates are A and E.

Step 2: Apply the degree heuristic

- Degree of A = 3.
- Degree of E = 2.

A has the larger degree, so choose **A**.

Step 3: Assign the first value

Values are tried in ascending order.

Assign:

$$\boxed{A = 1}$$

Step 4: Forward checking

A is connected to B, C and D. Remove slot 1 from each of their domains.

Exam	Domain before assignment	Domain after assigning A = 1
A	{1, 2}	{1}
B	{1, 2, 3}	{2, 3}
C	{1, 2, 3}	{2, 3}
D	{1, 2, 3}	{2, 3}
E	{1, 2}	{1, 2}

E remains unchanged because it is not directly connected to A.

(c) Continue the CSP search

Step 1: Assign A = 1

Current domains:

- B = {2, 3}
- C = {2, 3}
- D = {2, 3}
- E = {1, 2}

Step 2: Choose the next variable

MRV selects E because its domain has two values, compared with two values each for B, C and D.

The tied candidates are B, C, D and E. The degree heuristic chooses B, C or D because each has degree 3, compared with E's degree 2. Alphabetical order selects B.

Assign:

$$\boxed{B = 2}$$

Forward checking removes 2 from C and D because B is connected to both.

Updated domains:

Exam	Domain
A	{1}
B	{2}
C	{3}
D	{3}
E	{1, 2}

Step 3: Choose the next variable

C and D both have one remaining value, so MRV selects either. Their degrees are equal, so alphabetical order selects C.

Assign:

$$\boxed{C = 3}$$

Forward checking removes 3 from E? No—C and E must differ, but E has domain {1, 2}, so no value is removed.

D remains {3}, but C and D are **not connected by a constraint**, so they may share slot 3.

Updated domains:

Exam	Domain
A	{1}
B	{2}
C	{3}
D	{3}
E	{1, 2}

Step 4: Assign D = 3

D has only one value, so assign:

$$\boxed{D = 3}$$

Forward checking removes 3 from E, but E's domain is {1, 2}; therefore, its domain is unchanged.

Step 5: Assign E = 1

E has domain {1, 2}. By ascending order, try 1.

$$\boxed{E = 1}$$

All variables are now assigned.

Final assignment

Exam	Assigned slot
A	1
B	2
C	3
D	3
E	1

Verify all constraints

Constraint	Assigned values	Satisfied?
A–B	$1 \neq 2$	Yes
A–C	$1 \neq 3$	Yes
A–D	$1 \neq 3$	Yes
B–C	$2 \neq 3$	Yes
B–D	$2 \neq 3$	Yes
C–E	$3 \neq 1$	Yes
D–E	$3 \neq 1$	Yes

Final answer:

$$\boxed{A = 1, B = 2, C = 3, D = 3, E = 1}$$

All constraints are satisfied.

(d) How MRV, degree heuristic and forward checking help

Minimum Remaining Values (MRV)

MRV selects the exam with the fewest available slots. It focuses on the most constrained variable first, reducing the chance of reaching a dead end later.

Degree heuristic

The degree heuristic prioritizes an exam that constrains the largest number of unassigned exams. This helps resolve highly connected variables early.

Forward checking

Forward checking removes values from neighboring variables' domains whenever an exam is assigned. This reveals conflicts before the entire timetable is constructed.

If a domain becomes empty, the current partial assignment cannot be extended to a valid solution. The algorithm must backtrack and try another value for an earlier assignment.

Question 4: Board-Game AI – Minimax, Alpha-Beta Pruning and Evaluation Function

Topic: Game playing, adversarial search, evaluation functions | 15 minutes

A game-playing agent (MAX) searches two plies ahead: MAX chooses one of three moves (B, C, D), then MIN replies with one of three moves. Positions at depth 2 are scored by the evaluation function

$$\textbf{Eval} = 2 \times \textbf{M} + 1 \times \textbf{Mo}$$

where M is the material advantage of MAX and Mo is the mobility advantage of MAX. The leaf features (M, Mo) are given for the leaves in left-to-right order:

MIN node	Leaf 1 (M, Mo)	Leaf 2 (M, Mo)	Leaf 3 (M, Mo)
B	(1, 1)	(5, 2)	(3, 2)
C	(1, 0)	(1, 2)	(2, 2)
D	(6, 2)	(2, 1)	(0, 2)

(a) Compute the Eval value of all nine leaves.

(b) Run plain minimax. Give the backed-up value of B, C and D, the value at the root and the move MAX should play.

(c) Run alpha-beta pruning on the same tree (left to right). Show α and β at each node, mark every pruned branch, and state the root value and the number of leaves evaluated.

(d) If the leaves under C were ordered (6, 4, 2) instead, how many leaves would alpha-beta evaluate? Explain the effect of move ordering and why pruning never changes the minimax value.

(a) Calculate all nine leaf evaluation values

Use:

$$\text{Eval} = 2M + Mo$$

Node B

Leaf	M	Mo	Calculation	Eval
B1	1	1	2(1) + 1	3
B2	5	2	2(5) + 2	12
B3	3	2	2(3) + 2	8

Node C

Leaf	M	Mo	Calculation	Eval
C1	1	0	2(1) + 0	2
C2	1	2	2(1) + 2	4

Leaf	M	Mo	Calculation	Eval
C3	2	2	2(2) + 2	6

Node D

Leaf	M	Mo	Calculation	Eval
D1	6	2	2(6) + 2	14
D2	2	1	2(2) + 1	5
D3	0	2	2(0) + 2	2

Final leaf values:

- B: **3, 12, 8**
- C: **2, 4, 6**
- D: **14, 5, 2**

(b) Plain Minimax algorithm

MAX selects the move with the highest value. MIN selects the lowest value among its replies.

Calculate the value of B

$$B = \min(3, 12, 8) = \boxed{3}$$

Calculate the value of C

$$C = \min(2, 4, 6) = \boxed{2}$$

Calculate the value of D

$$D = \min(14, 5, 2) = \boxed{2}$$

Root value

MAX chooses the maximum among B, C and D:

$$V(\text{root}) = \max(3, 2, 2)$$
$$\boxed{V(\text{root}) = 3}$$

Best move for MAX: B

(c) Alpha-Beta Pruning

Alpha-beta pruning eliminates branches that cannot affect the final minimax decision.

Initial values:

$$\alpha = -\infty,\ \beta = +\infty$$

Evaluate the tree from left to right.

Step 1: Explore B

B is a MIN node.

- Evaluate B1 = 3.
- Update $\beta_B = \min(+, \infty 3) = 3$.
- Evaluate B2 = 12.
- Update $\beta_B = \min(3, 12) = 3$.
- Evaluate B3 = 8.
- Update $\beta_B = \min(3, 8) = 3$.

Therefore:

$$B = 3$$

At the root:

$$\alpha = \max(-, \infty 3) = 3$$

No pruning occurs at B because all three leaves have already been evaluated.

Step 2: Explore C

C is a MIN node with $\alpha = 3$.

- Evaluate C1 = 2.
- Update $\beta_C = \min(+, \infty 2) = 2$.

Now:

$$\beta_C = 2 \leq \alpha = 3$$

Therefore, **prune C2 and C3**. They cannot change the root's decision.

The backed-up value of C is 2.

Step 3: Explore D

D is a MIN node with $\alpha = 3$.

- Evaluate D1 = 14.
- Update $\beta_D = 14$.
- Evaluate D2 = 5.
- Update $\beta_D = \min(14, 5) = 5$.
- Evaluate D3 = 2.
- Update $\beta_D = \min(5, 2) = 2$.

The final value of D is:

$$D = 2$$

Alpha-beta summary

Node	Leaf evaluations	Backed-up value	Pruned leaves
B	3, 12, 8	3	None
C	2	2	C2 and C3
D	14, 5, 2	2	None

Total leaves in the tree:

$$3 + 3 + 3 = 9$$

Leaves evaluated:

$$3 + 1 + 3 = \boxed{7}$$

Leaves pruned:

$$\boxed{2}$$

Final answer:

- Root value = **3**
- Best move = **B**
- Leaves evaluated = **7**
- Leaves pruned = **2**

(d) If C's leaves are ordered (6, 4, 2)

The root has already explored B, so:

$$\alpha = 3$$

At MIN node C:

1. First leaf = 6; therefore, $\beta = 6$.
2. Second leaf = 4; therefore, $\beta = 4$.
3. Third leaf = 2; therefore, $\beta = 2$.

After the third leaf:

$$\beta = 2 \leq \alpha = 3$$

However, this condition is only reached after evaluating all three leaves.

Therefore, **all 3 leaves under C must be evaluated**.

The leaf counts become:

Node	Leaves evaluated
B	3
C	3
D	3
Total	**9**

Answer: 9 leaves are evaluated.

Effect of move ordering

Alpha-beta pruning is more effective when promising moves are examined first. A poor move ordering can delay pruning, requiring more leaves to be evaluated.

Why pruning does not change the minimax value

Alpha-beta pruning eliminates branches that cannot affect the final choice at the root. It changes the amount of computation, not the minimax result.

Therefore, the root value remains:

$$\boxed{3}$$

MAX still chooses move **B**.